Credit Card Fraud Detection

Detecting Fraudulent Card Transactions on a Highly Imbalanced Dataset

EDA → Feature Engineering → Model Comparison → Tuning → SHAP → Business Threshold

0.873	90%	50.3%
Best PR-AUC (XGBoost)	Recall at operating threshold	Precision at operating threshold

Muhammad Tahir Riaz

github.com/triaz-malik/Machine-Learning

Executive Summary

We built a fraud-detection system on roughly 1.85 million card transactions where only **0.58%** are fraudulent. Because the data is so imbalanced, accuracy is meaningless — a model that always predicts “not fraud” scores 99.4% accuracy while catching zero fraud. We therefore optimise **recall**, **precision**, and **PR-AUC** instead.

Moving from a linear baseline to a tuned tree model lifted PR-AUC from **0.18** to **0.87** (XGBoost). At a business-chosen operating point the model catches **90% of fraud** at **50.3% precision** — flagging only ~0.35% of legitimate transactions for review. Per 10,000 fraud cases this turns roughly 5,840 caught frauds (baseline) into ~9,000.

1. Business Problem

Banks must decide, in real time, whether to approve or block each card transaction. Fraud is rare but expensive, and the cost structure is **asymmetric**:

- **Missed fraud (false negative)** — direct financial loss and chargebacks.
- **False alarm (false positive)** — manual-review cost and customer friction.

The objective is to maximise the share of fraud caught (**recall**) while keeping false alarms low enough that the review team can cope (**precision**).

Why PR-AUC, not accuracy

With a 0.58% positive rate, accuracy and even ROC-AUC are misleadingly optimistic — both are dominated by the huge negative class. The **precision–recall curve** focuses entirely on the rare fraud class, so **PR-AUC** is the honest headline metric for this problem and the one we tune and rank models on.

2. Dataset & Class Imbalance

Dataset: simulated credit-card transactions (Sparkov), pre-split into train and test files. Fraud is a tiny fraction of all activity:

Split	Transactions	Fraud (rate)
Train	1,296,675	7,506 (0.579%)
Test (held-out)	555,719	2,145 (0.386%)

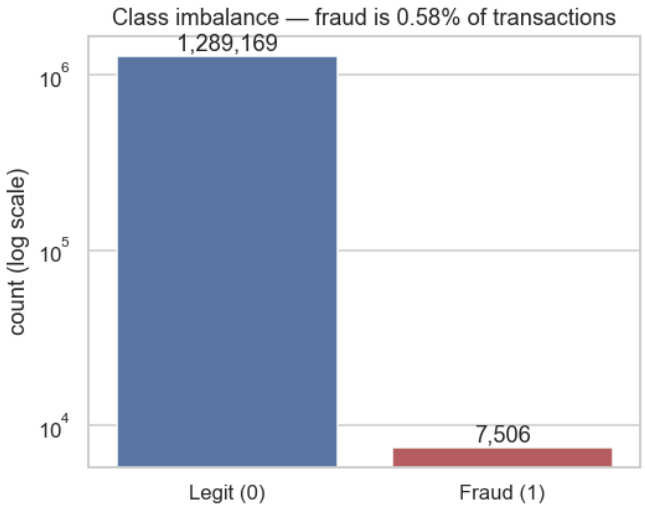

Fig 1. Extreme class imbalance — fraud is 0.58% of all transactions (log scale).

3. Exploratory Data Analysis

Profiling the full training set surfaced three dominant fraud signals, each later confirmed by the model:

3.1 Amount

Fraudulent transactions skew to far higher amounts — mean **$531** vs **$68** for legitimate transactions. Amount (and its log) is the single strongest signal.

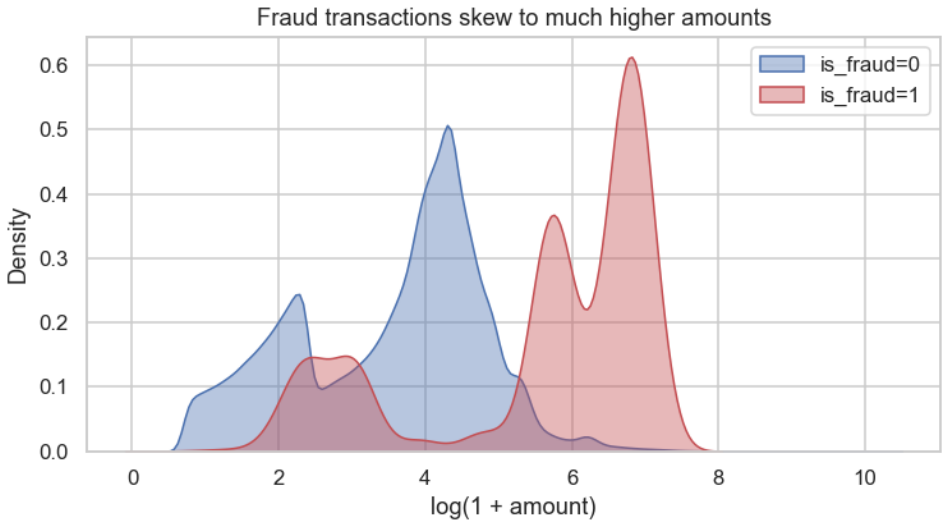

Fig 2. Transaction amount by class — fraud skews high.

3.2 Time of day

Fraud is a night-time phenomenon: rates spike between **22:00 and 03:59**, up to ~25× the daytime rate.

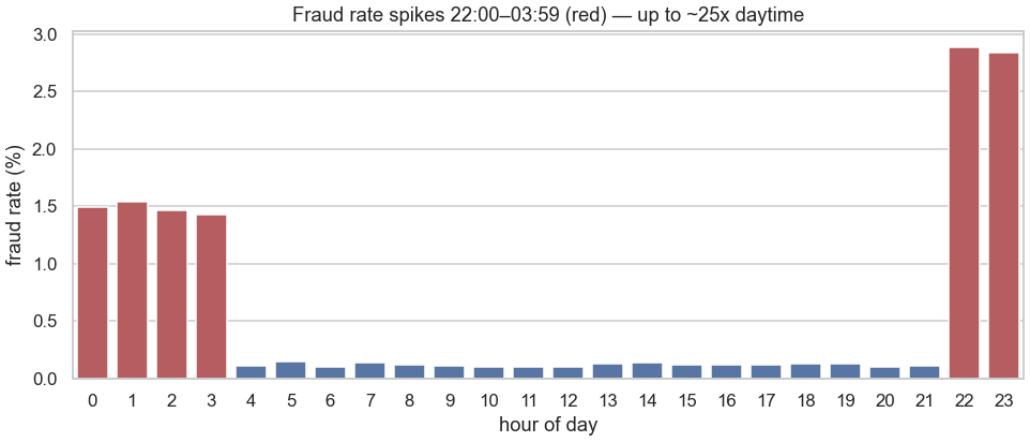

Fig 3. Fraud rate by hour of day.

3.3 Merchant category

Online categories (**shopping_net**, **misc_net**) carry the highest fraud rates — in-person categories are far safer.

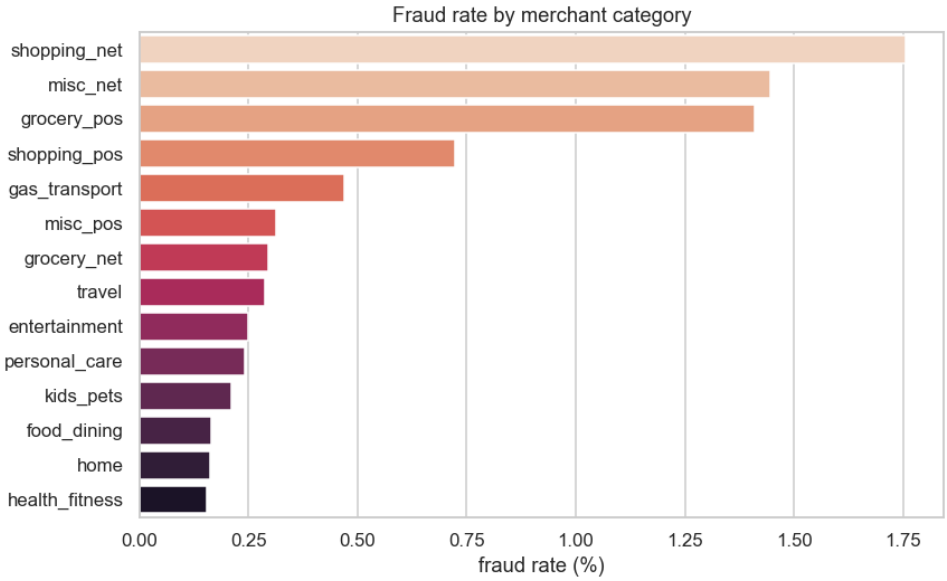

Fig 4. Fraud rate by merchant category.

3.4 A signal that was *not* there

Home–merchant geographic distance is a common fraud heuristic, but profiling showed both classes average ~76 km from home. It carries **no signal** on this simulated data, so it was dropped — verified, not assumed.

4. Feature Engineering

From the raw columns we engineered a leak-safe feature set (`src/features.py`). All transforms are pure / row-wise so there is no train→test leakage. PII and identifiers were dropped so the model cannot memorise individuals; leakage and non-predictive columns were removed.

Feature group	Details / rationale
Amount	amt and log_amt — the single strongest signal.
Time of day	hour and is_night (22:00–03:59), where fraud concentrates.
Calendar	day_of_week, is_weekend.
Demographic	age (from dob), gender_M, log city population.
Merchant category	category — one-hot encoded.
Dropped — PII	first, last, street, trans_num, raw cc_num, raw dob.
Dropped — leakage	unix_time (duplicates the timestamp).
Dropped — no signal	home–merchant distance (both classes ~76 km).

Class imbalance itself was handled at training time with **class weights** (Logistic Regression, Random Forest) and **scale_pos_weight** (XGBoost), not by discarding data.

5. Modeling & Comparison

We trained an explainable baseline, then progressively stronger models — each evaluated on the **same held-out test set** with the same metrics.

5.1 Baseline (Logistic Regression)

The linear baseline establishes a floor and remains fully interpretable via its coefficients, but cannot represent the feature interactions that define fraud.

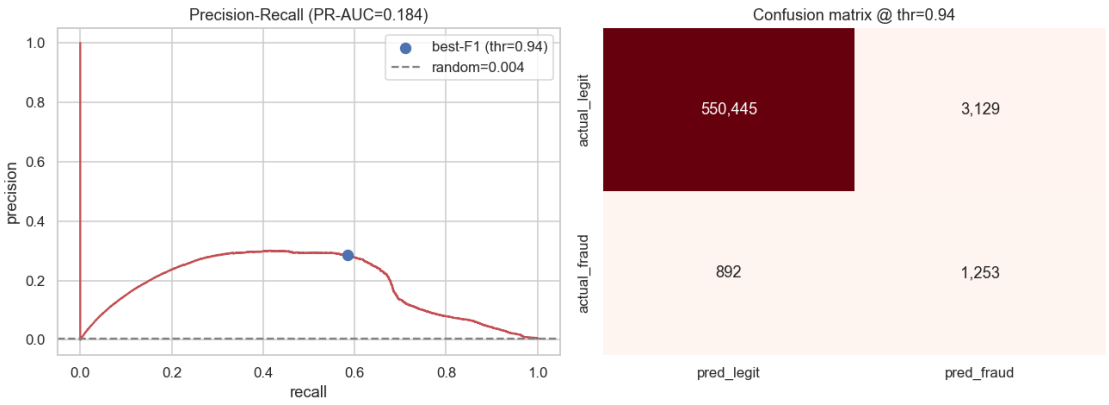

Fig 5. Baseline precision-recall curve and confusion matrix.

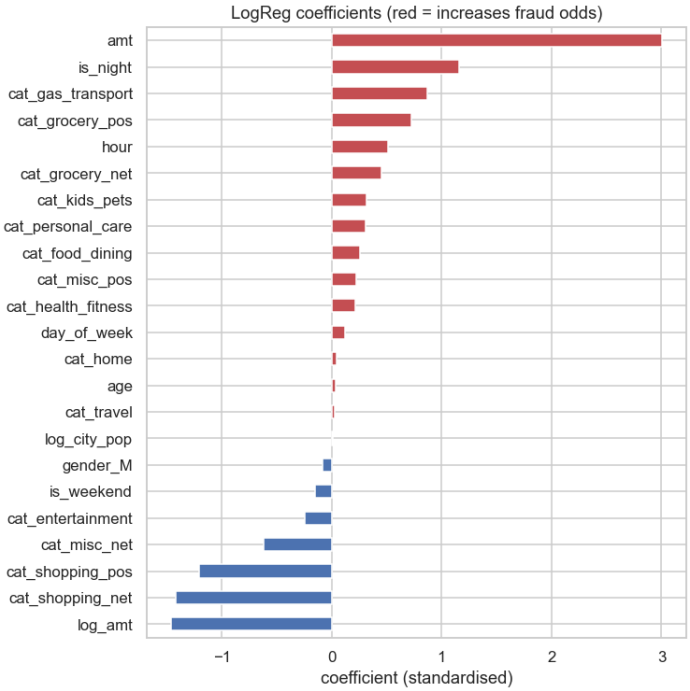

Fig 6. Baseline logistic-regression coefficients.

5.2 Model comparison

Model	PR-AUC	ROC-AUC	Precision	Recall	F1
LogReg (baseline)	0.184	0.964	0.286	0.584	0.384
RandomForest	0.872	0.997	0.896	0.747	0.815
XGBoost	**0.873**	**0.998**	**0.898**	**0.763**	**0.825**

PR-AUC is the right metric for imbalanced fraud. It leaps from **0.184** (linear baseline) to **0.873** (XGBoost) once we move to tree models. Note ROC-AUC is already ~0.96+ for every model — a vivid reminder that ROC-AUC flatters models on imbalanced data while PR-AUC reveals the real gap.

Why trees win: fraud lives in **interactions** — high amount *and* night *and* an online category. A linear model cannot represent these; gradient-boosted trees can.

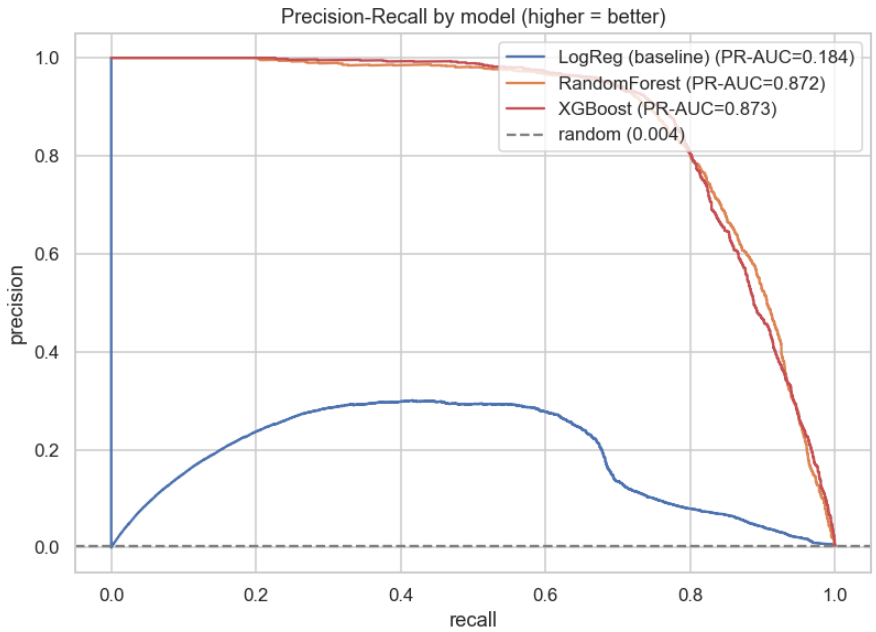

Fig 7. Precision-recall curves: tree models dominate across the whole range.

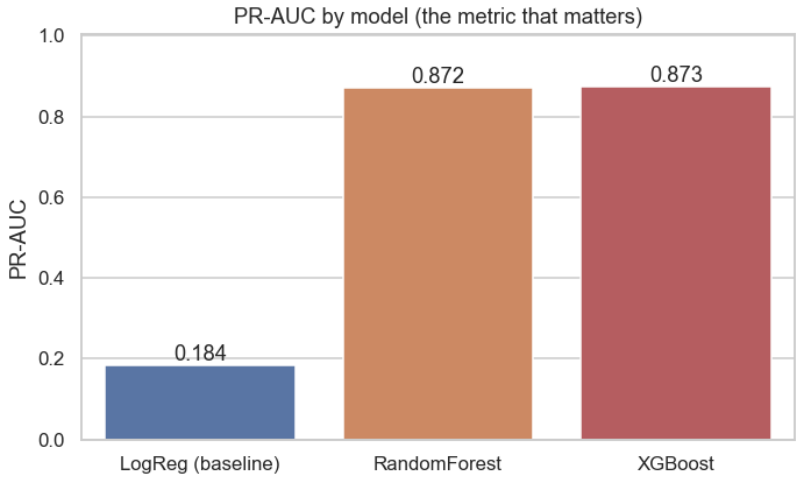

Fig 8. PR-AUC by model.

Tuning: a PR-AUC-scored RandomizedSearchCV for XGBoost selected max_depth=6, learning_rate=0.05, n_estimators=600, subsample=0.7, gamma=1 (CV PR-AUC 0.90).

6. Threshold Selection

Probabilities still need an operating threshold. The bank policy is simple: **catch at least 90% of fraud**. We pick the highest-precision probability threshold that meets that recall and read off the operational trade-off on the held-out test set.

Operating point	Value
Decision threshold	0.7999
Target recall (policy)	90%
Achieved recall	90.0%
Precision at threshold	50.3%
PR-AUC (tuned model)	0.880

The precision–recall trade-off is explicit: pushing recall to 90% drops precision to ~50%, meaning roughly half of all flags are genuine fraud. On 553,574 legitimate test transactions that is only ~0.35% sent for review — a volume the review team can absorb.

Outcome at the chosen threshold

Outcome at the 90%-recall threshold	Result
Frauds caught (true positives)	1,931 of 2,145 — 90.0%
Frauds missed (false negatives)	214 — 10.0%
False alarms sent to review	1,910 of 553,574 legit — 0.35%
Precision (flags that are real fraud)	50.3%

7. Explainability (SHAP)

The model is **auditable, not a black box**. SHAP confirms the EDA story: amount and night-time dominate, followed by hour and online categories.

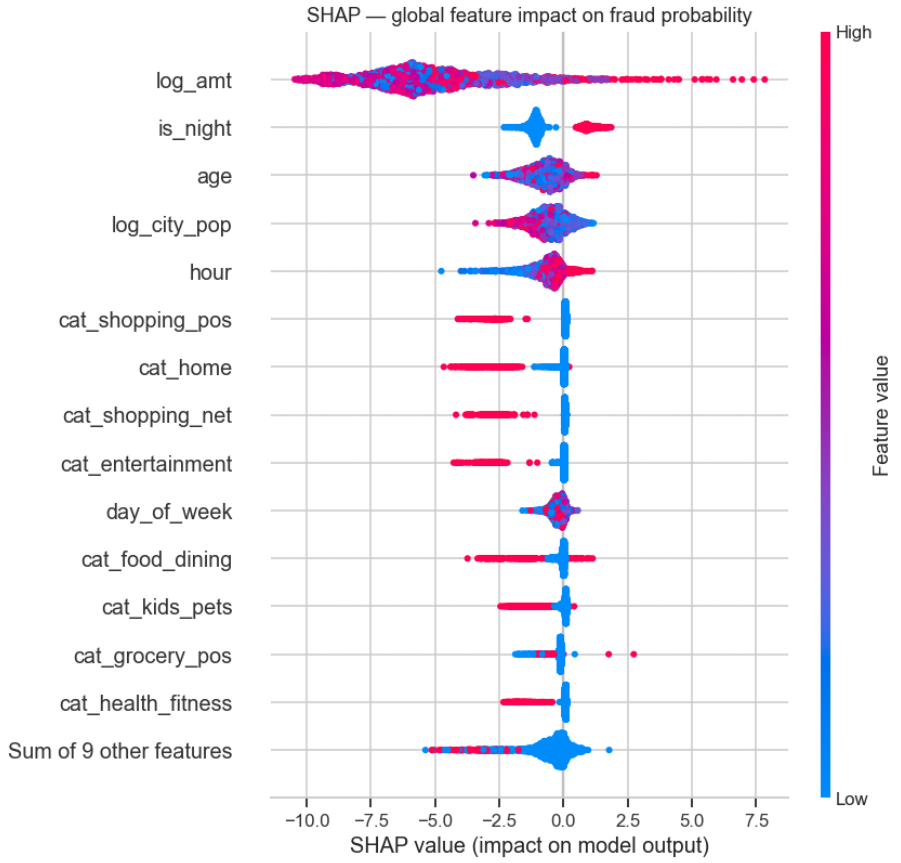

Fig 9. Global SHAP (beeswarm) — log_amt and is_night are the top drivers of fraud probability.

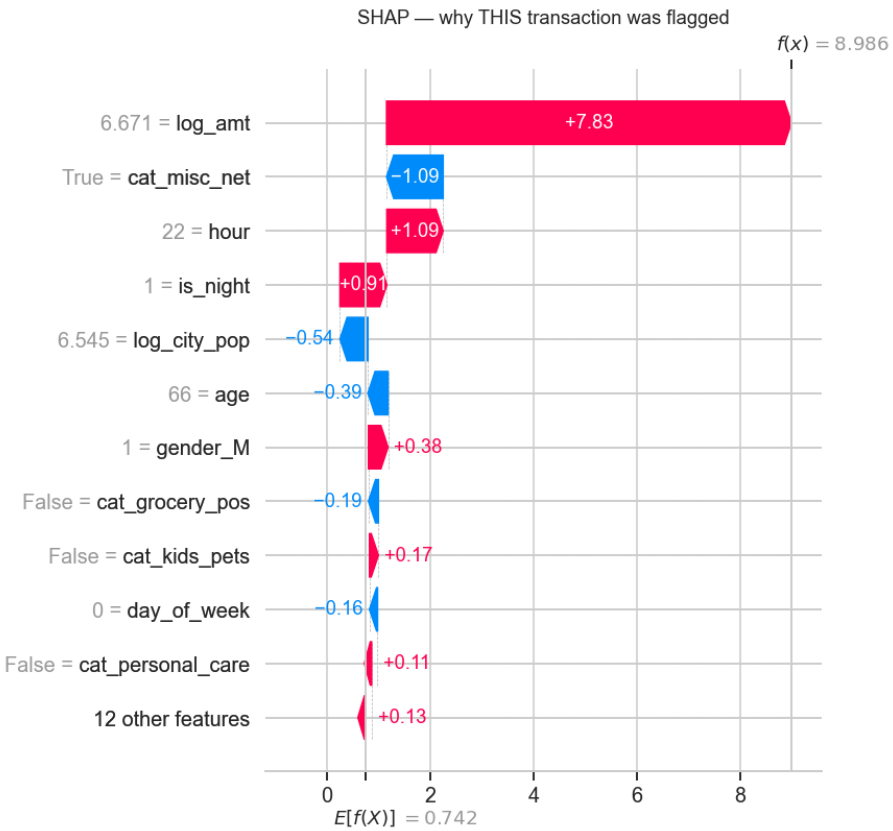

Fig 10. Local SHAP (waterfall) — why one specific transaction was flagged as fraud.

8. Business Impact & Future Work

Translated to scale — per 10,000 fraud cases:

- **Baseline Logistic Regression (58% recall):** catches ~5,840, misses ~4,160.
- **Tuned XGBoost (90% recall):** catches ~9,000, misses ~1,000.

Net impact: missed fraud falls by ~75%, while only 0.35% of legitimate customers experience a review — a strong recall gain at a manageable false-alarm cost.

Key findings

- **Amount** is the strongest signal — fraud averages $531 vs $68.
- **Time of day** matters enormously — fraud spikes 22:00–03:59.
- **Online categories** (shopping_net, misc_net) carry the highest fraud rates.
- **Geo-distance is not predictive** here — verified on the data, not assumed.
- **SHAP confirms the EDA** — log_amt and is_night are the top model drivers.

Future work

- **Cost-sensitive threshold** driven by real chargeback vs review costs, not a fixed 90%-recall policy.
- **Velocity / sequence features** (transactions per card per hour) to catch bursty fraud rings.
- **Probability calibration** so flagged scores map to real fraud likelihoods for triage.
- **Real-time deployment** behind a scoring API with monitoring for drift and concept change.

Generated from project artifacts in `reports/`. Metrics sourced from model_comparison.csv and threshold.json; figures from reports/figures/.